

Task 1: Language Models for Robot Control and Learned Legged Locomotion

1. LLM Approaches for Robot Control

Large language models (LLMs) can provide an interface between natural-language instructions and robot control systems. Current work uses LLMs for command understanding, task planning, and action selection in robotics [1]. For this MiniLab, three representative approaches are considered: structured-output command parsing, layered task planning, and affordance-based planning.

Structured-output command parsing. The LLM converts a user instruction into a predefined machine-readable format, such as JSON or a function call. For example, "walk forward for three seconds, then turn back" can be converted into an ordered move and turn sequence. The robot then executes these actions using predefined local skills. Related work has used LLMs for semantic task understanding, command parsing, and task allocation in robot systems [2]. This approach is also used in this MiniLab: the LLM handles language understanding, while local functions execute move, turn, and other commands.

Layered task planning. For more complex tasks, planning and execution can be separated into different layers. An LLM can handle high-level task decomposition while lower-level controllers execute individual robot skills. Layered LLM architectures have been studied for robot task planning and execution [3]. Language can also support later corrections and reuse of task knowledge; language corrections have been used to obtain generalizable knowledge for robot manipulation [4].

Affordance-based planning. A language instruction may describe an action that is not feasible for the robot in its current state. Affordance-based methods therefore combine language reasoning with information about the environment and the robot's available actions. Multimodal LLM reasoning has, for example, been combined with visual affordances for robotic grasping [5]. This type of approach is useful when action selection depends strongly on visual observations and physical feasibility.

Approach	Main LLM role	Output	Latency / cost	Typical use
Structured-output parsing	Interpret user commands	JSON / function calls	Relatively low; usually invoked per user command	Robots with a fixed skill set
Layered task planning	Decompose and plan tasks	High-level task sequence	Higher when multiple planning stages are required	Multi-step robot tasks
Affordance-based planning	Select feasible actions using context	Skills / action targets	Additional perception and reasoning are required	Manipulation and service robots

2. Learned Quadruped Locomotion

The MiniLab does not require training a walking policy from scratch. The recommended platform already provides a pretrained reinforcement-learning (RL) locomotion policy. RL has been used to learn robust quadruped locomotion [6], and more recent work has studied generalist policies that support multiple locomotion behaviors [7]. In this MiniLab, the upper-level program supplies velocity commands (v_x , v_y , w_z), representing forward/backward motion, lateral motion, and turning. The learned policy converts these commands and the robot state into locomotion control. The same general idea also supports policy transfer: sim-to-sim transfers a learned policy between simulators, while sim-to-real transfers a simulation-trained policy to a physical robot.

3. Two-Rate Architecture

LLM reasoning and low-level robot control operate at very different time scales. Real-time robot control requires frequent and predictable updates [8], whereas a cloud LLM is better suited to occasional high-level reasoning. The MiniLab therefore uses a two-rate architecture: the LLM parses a user command and produces a high-level action, while perception, motion skills, and locomotion remain in the fast local loop. On the recommended platform, MuJoCo physics runs at 200 Hz and the ONNX locomotion policy at 50 Hz, so the policy is updated about every 20 ms. The LLM does not need to run at this rate; it is called when a new user command must be interpreted.

References

- [1] Z. Qi and X. Jing, "Advances in Large Language Models for Robotics," 2024 7th International Conference on Mechatronics, Robotics and Automation (ICMRA), Wuhan, China, 2024, pp. 72-76, doi: 10.1109/ICMRA62519.2024.10809099.
- [2] S. Wu et al., "A Natural Semantic Task Understanding and Scheduling System for Crowd Robots," 2023 IEEE Smart World Congress (SWC), Portsmouth, United Kingdom, 2023, pp. 1-8, doi: 10.1109/SWC57546.2023.10448684.
- [3] K. Saketh, K. Jaideep, K. J. Goud, S. Farhat, M. A. Jabbar and A. Haqiq, "Layered Large Language Model Architectures for Robot Task Planning and Execution," 2026 3rd International Conference on Connected Innovation and Technology (ICCITX), Paris, France, 2026, pp. 219-225, doi: 10.1109/ICCITX70146.2026.11679896.
- [4] L. Zha et al., "Distilling and Retrieving Generalizable Knowledge for Robot Manipulation via Language Corrections," 2024 IEEE International Conference on Robotics and Automation (ICRA), Yokohama, Japan, 2024, pp. 15172-15179, doi: 10.1109/ICRA57147.2024.10610455.
- [5] Z. Zhao, J. Gao and D. Zheng, "Affordance-Guided Robotic Grasping via Multimodal Large Language Model Reasoning," IEEE Transactions on Automation Science and Engineering, vol. 23, pp. 4088-4100, 2026, doi: 10.1109/TASE.2026.3651854.
- [6] B. McAteer and K. Mason, "Robust Quadruped Locomotion via Evolutionary Reinforcement Learning," 2026 11th International Conference on Control and Robotics Engineering (ICCRE), Kyoto, Japan, 2026, pp. 84-88, doi: 10.1109/ICCRE69951.2026.11593505.
- [7] Y. Chen et al., "A Generalist Agent Learning Architecture for Versatile Quadruped Locomotion," IEEE Transactions on Automation Science and Engineering, vol. 22, pp. 22112-22123, 2025, doi: 10.1109/TASE.2025.3612147.
- [8] Q. Xi, C. W. Zheng, M. Y. Yao, W. Kou and S. L. Kuang, "Design of a Real-time Robot Control System oriented for Human-Robot Cooperation," 2021 International Conference on Artificial Intelligence and Electromechanical Automation (AIEA), Guangzhou, China, 2021, pp. 23-29, doi: 10.1109/AIEA53260.2021.00013.